Multi-Agent Orchestration Platform — Design & Decisions

Oct 14, 2026 · @Bishnu

Evaluation and verdict

Verdict: buildable as a ~10-week solo prototype, provided agents are configurations on one shared runtime and the master node is mostly deterministic code with one LLM supervisor on top. Building every listed feature to production quality in that time is not realistic, so video, self-learning skills and multi-organisation tenancy move to later phases.

The valuable core is a single knowledge layer plus a single task ledger shared by heterogeneous models, local and hosted. Most agent frameworks give you agents and tool loops; shared, resumable, team-scoped state is the part they leave to you, and it is what makes this project worth building.

Risk area	Why it is hard	How this design handles it
Shared context and hand-off	RAG retrieves facts, but an agent cannot resume a task from facts alone; it needs the plan, progress and decisions	A structured task ledger, an append-only event log and graph checkpoints (Shared knowledge)
Automatic routing	LLM routers misroute when agent descriptions overlap	Capability cards, a two-stage router, a confidence threshold that asks the user, and a routing test set
Tool and content security	MCP servers and scraped pages are prompt-injection channels into agents that share one database	Least-privilege tool grants, untrusted-content tagging, sandboxed execution, human approval for side effects
Local model quality	Small Ollama models are weaker at multi-step tool calls and strict JSON	Per-agent model choice; tool-heavy and supervisor roles go to stronger models
Cost across providers	Each API reports usage differently; local models report none	One model gateway that meters every call

Risk area	Why it is hard	How this design handles it
Supervisor thrash	An LLM supervising LLMs can loop, re-delegate or stall	Step, token, time and money budgets enforced in code, not in prompts

Two points in the brief need adjusting:

- **One database, scoped context.** Sharing one database is right for storage. Sharing all of it with every agent is wrong: prompts fill with irrelevant memory and team data leaks across scopes. Agents read through scoped retrieval.
- **Split the supervisor's job.** Scheduling, retries and budget limits must be code so they behave the same every time. The LLM supervisor plans, reviews, corrects and reassigns.

Prototype scope and assumptions

The prototype (v0.1) covers 15 of the 18 requested capabilities end to end; video, the messaging gateway and enterprise SSO wait for v0.2.

Capability from the brief	v0.1 prototype	Later (v0.2+)
Master node: control and monitor agents	Supervisor graph, router, scheduler, live monitor	Multi-node workers, autoscaling
Agent backends	Ollama in Docker plus two hosted APIs through one gateway	vLLM for higher local throughput
Shared knowledge database	One PostgreSQL with pgvector, hybrid search	Dedicated vector store if chunks exceed ~10M
Skills and mastery domains per agent	Skill folders plus a capability card per agent	Agents proposing new skills for human review
MCP servers	Registry, per-agent allowlist, stdio and HTTP	Public MCP registry browsing, OAuth per user
Team context and instruction pool	Org, team, project and agent instruction layers	Instruction A/B tests
Manual and automatic routing	`@agent` mentions and a two-stage router	Learned router from feedback

Capability from the brief	v0.1 prototype	Later (v0.2+)
Verification agents	Code checks in a sandbox plus an LLM reviewer; citation checks for research	Formal property tests, mutation testing
Diagrams	Mermaid compile-and-repair loop	Other diagram engines
Audio, image, video	ElevenLabs speech, transcription and image	ElevenLabs video, music, dubbing
Web scraping with quality filtering	Crawl, extract, dedupe, score, store with provenance	Scheduled source monitoring
RAG, context pool, resumable tasks	Task ledger, event log, checkpoints, scoped retrieval	Cross-task memory consolidation
Supervisor that corrects, delegates, schedules	LLM supervisor plus code-enforced budgets and cron	Self-tuning budgets
Interface	Web dashboard and a terminal client	Slack, Telegram and Discord gateway
Usage monitoring	Tokens, cost, latency per user, team, agent and task	Alerts, chargeback exports
Agent customisation	UI and YAML for model, skills, tools, instructions, limits	Versioned rollouts of agent changes
User system	Reuse your FastAPI multi-tenant JWT + RBAC platform	SSO through an identity provider
Security	Secrets encryption, tool grants, sandbox, approvals, audit log	External penetration test

Out of scope entirely: model fine-tuning, a public multi-tenant SaaS with billing, and an agent marketplace.

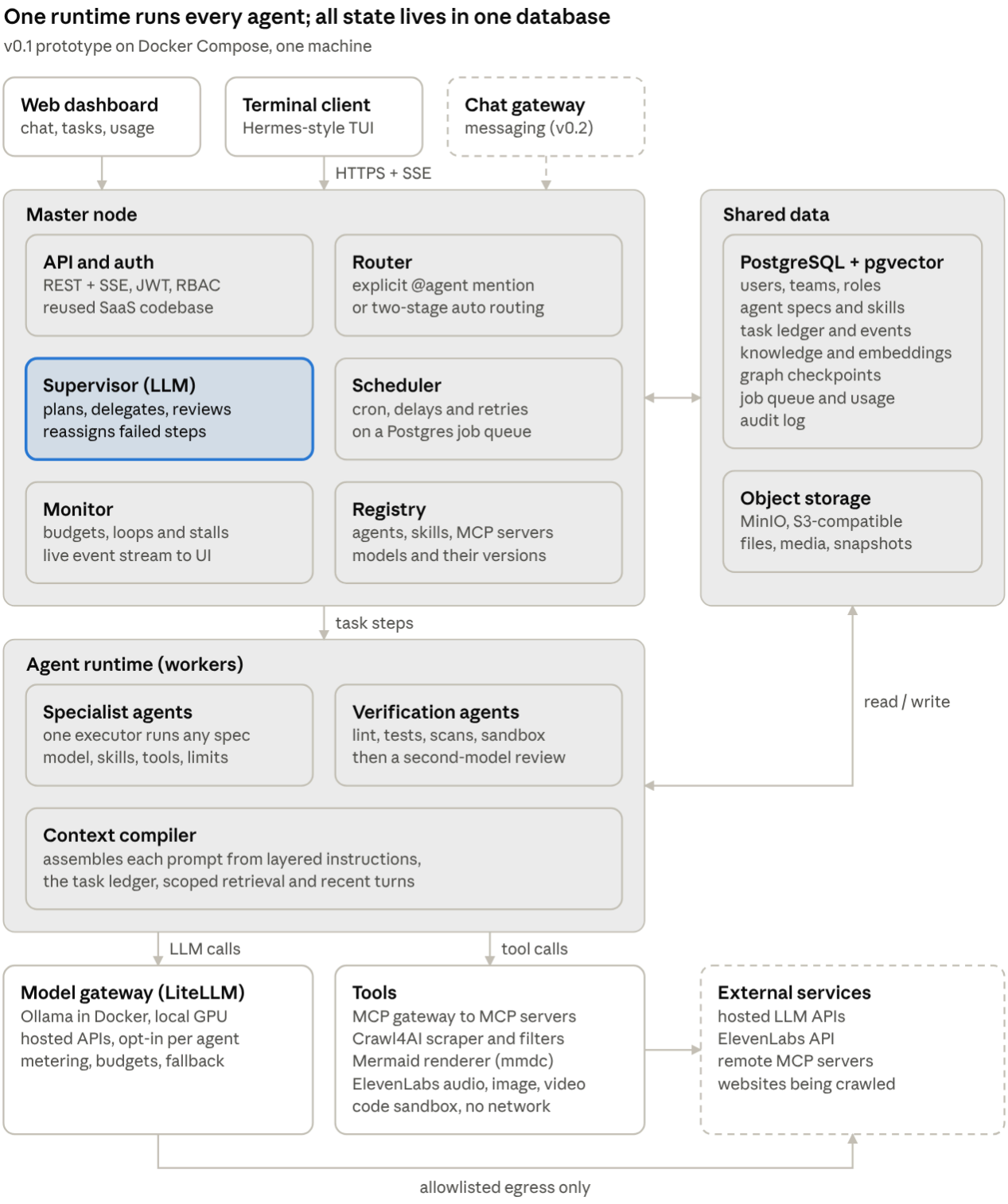
Assumptions this doc rests on (change any and the affected decisions are flagged in Decision records):

- One developer, about 10 weeks, Python backend.
- Self-hosted with Docker Compose on one Linux machine with an NVIDIA GPU of 16–24 GB VRAM. Without a GPU, local models are limited to small ones on CPU.
- 5–20 users across a few teams in a lab or small company, not public-internet scale.

- Some data is sensitive, so embeddings and default models run locally; hosted APIs are opt-in per agent.
- The existing FastAPI + PostgreSQL + JWT + RBAC multi-tenant codebase is reused for users, teams and roles.

System architecture

Four layers run on one machine — clients, the master node, a shared agent runtime, and models plus tools — all reading and writing one PostgreSQL database.



system architecture · 4 layers around one database

Clients talk only to the master node, which dispatches steps to one shared runtime; the highlighted supervisor is the only component that plans and reassigns work. Because the master and every worker share the same database, any worker can resume any task, and only the model gateway and tool layer reach outside. Dashed boxes are v0.2 or external.

Technology stack and decision records

The stack is Python, LangGraph, LiteLLM and a single PostgreSQL with pgvector; each choice favours fewer moving parts on one machine over peak scale. Every third-party piece sits behind a thin internal interface so it can be swapped without touching agents.

Area	Decision	Why	Rejected, and why
Backend	Python 3.12 + FastAPI, reusing your multi-tenant JWT + RBAC codebase	Matches the ML and scraping ecosystem; auth, tenancy and roles already exist	Node/TypeScript: weaker ML and scraping libraries for this team
Agent runtime	LangGraph 1.x behind an internal `AgentRuntime` interface	Postgres checkpoints make every step resumable; interrupts give human approval; longest production record (GA Oct 2025)	CrewAI: fastest start, less control of state. Microsoft Agent Framework: GA April 2026, younger, Azure-leaning. Own loop: re-implements checkpointing
Model access	LiteLLM proxy in front of every model	One OpenAI-compatible endpoint for Ollama and APIs; virtual keys with budgets per key, team and user; spend logged to Postgres	Direct provider SDKs: no central metering. Hosted routers: cannot front a local Ollama
Local inference	Ollama in Docker, internal network only	OpenAI-compatible `/v1` endpoints; tool calls and JSON-schema output on supported models	vLLM: higher throughput, heavier setup; revisit past ~5 concurrent users
Database	PostgreSQL 18 + pgvector 0.8	One source of truth for users, specs, ledger, events, knowledge, checkpoints, jobs and usage; row-level security scopes teams	Separate vector DB (Qdrant, Weaviate): a second store to secure and sync, no gain below ~10M chunks

Area	Decision	Why	Rejected, and why
Retrieval	Hybrid: Postgres full-text + vector, reciprocal-rank fusion, then a local reranker	Catches exact names, IDs and code symbols that embeddings miss	Vector-only search
Embeddings	Local model via Ollama (e.g. bge-m3, 1024 dims); model name stored per chunk	Sensitive text stays on the machine; a model change becomes a planned re-embed	Hosted embeddings: data egress, per-token cost
Jobs and cron	Procrastinate (Postgres-backed queue with periodic tasks, retries, locks)	No extra service; jobs commit in the same transaction as task rows	Celery + Redis: extra service. Temporal: strong, but duplicates LangGraph checkpoints and adds heavy ops
Tools	MCP, targeting the stateless 2026-07-28 revision via the official SDK	Any MCP server plugs in; stateless servers restart and scale cleanly	Framework-specific tool plugins: lock-in
Skills	Agent Skills format (a folder with `SKILL.md`)	Open standard; progressive disclosure keeps prompts small; skills port to other tools	Custom JSON prompt packs: no portability
Web scraping	Crawl4AI in Docker, Trafilatura as fallback extractor	Self-hosted, renders JavaScript, outputs LLM-ready markdown; v0.9 hardened its API server	Hosted crawlers: cost and data egress. Requests + BeautifulSoup: no JavaScript
Media	ElevenLabs API for speech, transcription, sound, image, video; mermaid-cli container for diagrams	One media vendor and one key; image and video are async with webhooks	Separate image and video vendors: more keys and billing

Area	Decision	Why	Rejected, and why
Code execution	Ephemeral Docker containers: no network, read-only root, non-root user, CPU, memory and time caps	Runs untrusted agent code locally at no cost	Hosted sandboxes: fine later, but code and data leave the machine
Interface	React + Vite dashboard and a Textual terminal client on one API	Mirrors Hermes: a terminal UI plus a dashboard, messaging gateway later	Streamlit or Gradio: weak multi-user auth and concurrent streams. Forking a chat UI: chat-only data model
Observability	OpenTelemetry traces + LiteLLM spend logs into own usage tables	No extra datastore in v0.1	Langfuse v3 now: needs ClickHouse, Redis and S3 beside Postgres; adopt in v0.2
Secrets	Provider and MCP credentials encrypted at rest (AES-GCM, key from a Docker secret), injected at call time	Keys never enter prompts, logs or agent context	Plain environment variables per agent
Agent-to-agent	Internal: via the task ledger. External: A2A v1 in v0.2	Ledger hand-offs are auditable and resumable; A2A is the Linux Foundation standard for exposing agents to other systems	Free-form agent chat: hard to audit, prone to loops

If an assumption changes: no GPU moves bulk work to a cheap hosted model; more than ~20 concurrent users adds Redis and vLLM; a strict no-egress rule drops hosted APIs and ElevenLabs.

Master node: routing, supervision and scheduling

The master node is five parts in one service: API, router, supervisor, scheduler and monitor. Only the router and supervisor call an LLM; scheduling, budgets, retries and kill switches are plain code.

Routing: manual or automatic

- **Manual.** `@agent-name` in a message, or `agent_id` in the API, skips the router. Permission and budget checks still apply.
- **Automatic, two stages.**
  1. *Shortlist.* Embed the request and match it against each agent's capability card (description, mastery tags, skill descriptions, 3–5 example tasks). Drop agents the user may not invoke or that lack the needed tools or data scope; keep the top 3.
  2. *Pick.* A strong model chooses one agent, or splits the request into subtasks, or asks the user. Output is schema-validated JSON: `agent_id`, `confidence`, `reason`, optional `subtasks`.
  3. *Threshold.* Below 0.6 confidence, or when the top two are close, the user gets a one-click choice between them.
- **Learning loop.** Every routing decision and user override is logged. Overrides become labelled cases in a routing test set; auto-routing becomes the default once it reaches 85% top-1 accuracy on 100 labelled tasks.

Supervisor: plan, delegate, review, correct

The supervisor is an LLM agent on the strongest available model, running as a LangGraph graph with Postgres checkpoints.

1. **Plan.** Turn the request into a task ledger: goal, acceptance criteria, steps, assigned agent per step, dependencies.
2. **Delegate.** Dispatch every step whose dependencies are met; independent steps run in parallel.
3. **Review.** After each step, compare output and verification results with the acceptance criteria, then accept, request a revision, reassign or escalate.
4. **Correct.** A rejected step gets a correction note in the ledger and up to 2 retries, then one reassignment to another capable agent, then a human.
5. **Schedule.** The supervisor may request a recurring or delayed task ("re-check these sources every Monday"); the scheduler stores and runs it.

Plans above a cost threshold, and any step with external side effects (sending, posting, deleting, paying), pause for human approval through a LangGraph interrupt.

Guards enforced in code

Guard	Default	Action when hit
Steps per task	30	Stop and escalate to the task owner

Guard	Default	Action when hit
Cost per task	Set per team, e.g. $2	Pause, ask owner to raise or cancel
Wall time per step	15 min	Retry once, then reassign
Identical tool call repeated	3 times	Halt the step as a loop
Delegation cycle (A → B → A)	2 cycles	Halt and escalate
Step with no events	5 min	Treat as stalled; retry from checkpoint
Kill switch	Admin action	Pause one agent, one team or everything

Scheduler

Schedules are rows (cron expression, task template, owner, next run). A Procrastinate periodic job checks due schedules every minute and enqueues them. A scheduled run executes with the permissions of the user who created it, never with elevated rights, and stops if that user loses access.

Agents: backends, skills and customisation

An agent is a versioned configuration record, the AgentSpec; one generic executor runs every spec, so adding or changing an agent never means deploying code.

```
id: code-reviewer
name: Code Reviewer
version: 3
description: Reviews Python and TypeScript changes for bugs, security and
style.
mastery: [python, typescript, security-review]
model:
  primary: strong-api        # LiteLLM alias -> a hosted model
  fallback: local-coder      # LiteLLM alias -> an Ollama model
  temperature: 0.2
instructions: |
  Report each issue with file, line, severity and a concrete fix.
skills: [python-review, owasp-checklist]
tools:
  mcp: [github.read_pr, github.comment]
  builtin: [sandbox.run_tests, knowledge.search]
knowledge_scopes: [team:platform, project:billing-api]
```

```
limits: {max_steps: 20, max_cost_usd: 1.50, timeout_s: 900}
approval_required: [github.comment]
verification: code-default
examples:
  - "Review PR 42 in billing-api"
  - "Check this diff for SQL injection"
```

Agents name LiteLLM aliases, not concrete models, so swapping a model is a gateway config change. The `description`, `mastery`, skill descriptions and `examples` form the capability card the router embeds.

Which backend for which role

Role	Default backend	Reason
Supervisor and router	Strongest hosted model	Planning and strict JSON output matter most here
Verification reviewer	A different model family from the author	Two families share fewer blind spots
Coding agents	Strong hosted model, or the largest local coder that fits VRAM	Multi-step tool-call reliability
Bulk work: summarising, extraction, scoring scraped pages	Local model on Ollama	High volume, sensitive text, no per-token cost
Embeddings and reranking	Local	Data stays on the machine

Open-weight models served locally are still less reliable at multi-step tool calls and nested JSON than top hosted models, so the executor validates every tool call against its schema and retries once with the validation error before failing over to the fallback alias.

Skills and mastery

Skills use the Agent Skills folder format. At step start, an agent sees only the name and description of each skill it is granted; it loads a full `SKILL.md` when the task matches. Skill scripts run only in the sandbox, and a new or edited skill stays disabled until an admin approves its content hash.

Customisation

- Form editor and raw YAML editor for the same spec; every save creates a new version with a diff and one-click rollback.
- A playground runs the draft version against saved example tasks before it goes live.
- Teams can clone a shared agent and override instructions, scopes and limits without touching the original.
- Model, tools and knowledge scopes can only be granted within what the editor's own role allows.

Shared knowledge: RAG, context pool and hand-off

Any agent can pick up any task because nothing essential lives in a model's chat history: context is split across five stores with different jobs, and each agent's prompt is rebuilt from them at every step.

Store	Holds	Written by	Read by	Lifetime
Instruction pool	Org, team, project and agent instructions, versioned	Admins and team leads	Context compiler, every step	Until changed
Task ledger	Goal, acceptance criteria, plan, step status, decisions, open questions, artifact links	Supervisor and agents, through typed updates only	Any agent taking a step	Life of the task, then archived
Event log	Every message, tool call, result, routing decision and cost; append-only	Runtime	Monitor, UI, audit, replay	Retention policy, e.g. 90 days

Store	Holds	Written by	Read by	Lifetime
Knowledge base	Documents, scraped pages, code, finished-task summaries as chunks with embeddings, provenance and trust score	Ingestion pipeline; agents via a reviewed `remember` tool	`knowledge.search`, scoped to the caller	Until deleted or superseded
Checkpoints	LangGraph state after each node	Runtime	Runtime on resume	Life of the task

Context compiler

Every LLM call gets a prompt assembled in this order, each part capped as a share of the model's context window:

1. Agent instructions and the index of granted skills (~10%).
2. Instruction pool layers, org → team → project; later layers override earlier ones, and conflicts are flagged in the UI (~10%).
3. Ledger summary: goal, criteria, current step, decisions so far (~15%).
4. Retrieved knowledge from permitted scopes, each chunk wrapped as untrusted data with its source and trust score (~45%).
5. Recent turns of this step; older turns collapse into a rolling summary (~20%).

Small-context local models simply receive fewer chunks; the order never changes.

Resume protocol

1. A worker claims a ready step with a row lock.
2. It loads the step's latest checkpoint; with none, it starts the step from the ledger.
3. It reads the ledger and the step's last 50 events, writes a `resumed_by` event, and continues.

The same path handles a crashed worker, a reassignment and a human taking over, which is why the hand-off test is a v0.1 exit criterion.

Retrieval

- Ingestion parses PDF, DOCX, HTML, Markdown and code, chunks by structure (headings, functions) at ~500–800 tokens with overlap, embeds locally and stores `team_id`, `project_id`, source, fetch time, content hash, trust score and embedding model.
- Queries run full-text and vector search in parallel, fuse with reciprocal rank, rerank the top 50 locally and return the top 8 with citations.
- Row-level security on `team_id` means a query runs with the invoking user's team context, so an agent invoked by team A can never read team B's chunks.

Memory writes

Agents propose knowledge through `remember(fact, source)`. Proposals from verified sources or human uploads go live; everything else waits as pending until the supervisor or a person approves it. This blocks one compromised agent or poisoned page from rewriting shared memory, and every chunk can be rolled back by source.

Tools: MCP, web scraping, diagrams and media

All tool calls, built-in or MCP, pass through one tool gateway in the master node that checks grants, validates arguments, meters cost and writes the event log.

MCP servers

- **Registry.** Admins add servers as local containers (stdio) or remote URLs (HTTP). The current spec revision is 2026-07-28, which made the protocol core stateless and deprecated roots, sampling and logging (summary). The client negotiates versions so older servers still work.
- **Pinning.** On approval, each tool's name, description and schema are hashed. A changed hash disables that tool until an admin re-approves it, which blocks silent tool-description changes.
- **Grants.** Tools are granted per team and per agent, each flagged read-only or side-effecting. Side-effecting tools can require approval per call.
- **Credentials.** v0.1 uses service credentials stored encrypted; per-user OAuth for remote servers comes in v0.2.

Web scraping with a factual and usefulness filter

1. **Fetch** with Crawl4AI: robots.txt respected, per-domain rate limit, domain allow and deny lists.

2. **Extract** main content to markdown, dropping navigation, ads and boilerplate; Trafilatura re-extracts when Crawl4AI output is thin.
3. **Deduplicate** by exact hash, then near-duplicate fingerprint (MinHash).
4. **Cheap filters:** language, length, text-to-markup ratio, spam patterns.
5. **Score** with a local model into a fixed JSON schema: relevance to the request, information density, count of checkable claims, opinion versus fact, recency; combine with a maintained domain-reputation list.
6. **Check claims** for pages that pass: extract key claims and compare each against at least two independent sources or trusted knowledge, marking it corroborated, uncorroborated or contradicted.
7. **Store** pages above the threshold in the knowledge base with scores and provenance; the rest go to a quarantine table for audit.

No automatic filter can certify a page as true. Scores rank and flag; agents must cite sources, and the research verifier checks those citations.

Scraped text is the main prompt-injection channel. The agent that reads raw pages holds no side-effecting tools, and only its schema-validated extraction moves on to other agents.

Mermaid diagrams

An agent writes Mermaid source; the renderer runs mermaid-cli (`mmdc`) in a container. Invalid syntax makes `mmdc` exit non-zero with parse errors, which go back to the agent for up to 3 repair attempts. Source and SVG/PNG are stored together as an artifact.

Audio, image and video with ElevenLabs

- Speech synthesis, transcription, sound effects, music and dubbing use the standard ElevenLabs API.
- Image and video use the Image & Video API, which is asynchronous: submit, then receive a webhook or poll (images no faster than every 2 s, video every 10 s).
- It requires a Pro plan or above; lower tiers get a `402` error.
- Results arrive as a signed URL that expires about an hour later, so a worker downloads every result into object storage immediately.
- Failed generations are not charged. ByteDance models are off by default for API use and need explicit approval.
- Every media call shows an estimated credit cost first; video generation requires approval in v0.1 because it is the most expensive call.

Verification agents

Verification runs deterministic checks first and an LLM reviewer second; a step completes only when both pass or a person overrides. Deterministic checks go first because they are cheap, repeatable and cannot be talked out of a failure.

Code verification pipeline

1. **Static:** ruff for lint and format, pyright or mypy for types (eslint and tsc for TypeScript).
2. **Security:** semgrep and bandit rules, gitleaks for secrets, pip-audit or npm audit for dependencies.
3. **Tests:** existing tests plus tests the author agent wrote, run in the sandbox, with coverage of changed lines reported.
4. **LLM review:** a model from a different family than the author's reads the diff, the check results and the acceptance criteria, and returns JSON: `verdict`, then `issues` with file, line, severity and fix.
5. **Gate:** a failing test or any high-severity issue sends the issue list back to the author. After 2 rounds the supervisor reassigns the step or asks a person.

Other verifiers

Verifier	Checks	Fails when
Research	Every claim cites a stored source that exists and supports it	A claim is uncited, the source is missing, or a source contradicts it
Diagram	Mermaid compiles; labels match the request	`mmdc` exits non-zero after repairs
Structured data	Output validates against the step's JSON schema	Schema errors remain after one repair
Media	File exists, plays, duration and format match the request	Download failed or the spec does not match

Each agent declares a verification profile (`none`, `light`, `code-default`, `strict`). First-pass verification rate per agent is a headline metric on the monitoring dashboard, because it shows which agents need better instructions or models.

Users, teams and security

Agents never hold more rights than the person who invoked them; that one rule, enforced in the tool gateway and in Postgres row-level security, carries most of the security design.

Users and teams

One organisation contains teams; users belong to teams with one role each. A team shares an instruction pool, knowledge scopes, a task board, agent grants and a budget, which is what lets members invoke the same agents with the same context. Tasks are team-visible by default and can be marked private.

Role	Can
Owner	Everything, including provider keys, budgets and deleting the organisation
Admin	Manage agents, skills, MCP servers, tool grants, users and team budgets
Member	Invoke granted agents, create tasks and schedules, add team knowledge, approve actions on own tasks
Viewer	Read tasks, results, knowledge and dashboards

Controls mapped to OWASP agentic risks

The OWASP Top 10 for Agentic Applications 2026 is the checklist for this design.

Risk	Controls in v0.1
ASI01 Agent goal hijack	Retrieved, scraped and MCP content wrapped as untrusted data; quarantined reader agents with no side-effecting tools; approval for side effects
ASI02 Tool misuse	Per-agent allowlists, argument schema validation, per-tool rate limits, approval for destructive tools
ASI03 Identity and privilege abuse	Agents act with the invoking user's rights; short-lived scoped tokens; no shared admin credentials
ASI04 Agentic supply chain	Pinned MCP images and tool-description hashes; skills enabled only after review; dependency audit in CI
ASI05 Unexpected code execution	Code runs only in the sandbox: no network, no host mounts, resource caps

Risk	Controls in v0.1
ASI06 Memory and context poisoning	Reviewed memory writes; provenance and trust score on every chunk; rollback by source
ASI07 Insecure inter-agent communication	Hand-offs only through typed ledger updates, schema-validated and logged
ASI08 Cascading failures	Budgets, per-provider circuit breakers, capped retries
ASI09 Human–agent trust exploitation	Approvals show the exact action and arguments; answers show sources and verification status
ASI10 Rogue agents	Anomaly alerts from the monitor, kill switch, full audit log

Platform baseline

- TLS through a Caddy reverse proxy; only Caddy is exposed. Postgres, Ollama, LiteLLM, MinIO and MCP containers stay on the internal Docker network.
- Argon2 password hashing, short-lived access JWTs with rotating refresh tokens, CSRF protection, login rate limiting, API tokens for the terminal client.
- Outbound traffic from sandbox and tool containers goes through an egress proxy with a domain allowlist.
- Secrets encrypted at rest, never logged, never placed in prompts.
- Audit log of logins, permission changes, approvals, agent and tool changes.
- Nightly `pg_dump` plus object-storage sync, with a monthly restore test.

Interface and usage monitoring

The interface follows the Hermes Agent pattern — a terminal UI first, a browser dashboard beside it, chat apps later — with all three on one API so nothing works in one place and not another.

Web dashboard (v0.1)

Screen	What it does
Chat	`@agent` autocomplete, or auto-routing; streams each agent's steps, tool calls and verification live over SSE

Screen	What it does
Tasks	Board of team tasks; each opens its ledger, step timeline, artifacts and costs
Approvals	Inbox of paused actions showing the exact tool and arguments, approve or reject
Agents	List, form and YAML editor, version diff, playground, clone
Knowledge	Sources with trust scores, quarantine queue, pending memory proposals
Tools	MCP servers, pinned hashes, grants per team and agent
Schedules	Recurring and delayed tasks, last and next run
Usage	Cost, tokens and quality dashboards (below)
Admin	Users, teams, roles, provider keys, budgets, audit log

Terminal client (v0.1)

A Textual TUI plus plain commands for scripts, for example `hub chat`, `hub run @coder "fix the failing test"`, `hub tasks`, `hub approve <id>` and `hub logs --follow` (`hub` is a placeholder name). v0.2 adds a messaging gateway for Slack, Telegram and Discord.

Usage monitoring

Every LLM and tool call writes one usage record: user, team, agent, task, step, model, provider, input and output tokens, cost, latency, cache hit and status. Hosted-model cost comes from LiteLLM's spend tracking; local calls record tokens and GPU-seconds at zero cost.

The Usage screen shows:

- Cost and tokens over time, split by team, agent and model.
- Budget burn against limits, with alerts at 80% and a hard stop at 100%.
- Most expensive tasks and agents this week.
- Quality: first-pass verification rate, retry rate, routing override rate and task success rate per agent.
- Latency at p50 and p95 per model and per tool.

Budgets are enforced twice: LiteLLM virtual keys per team and agent stop spend at the gateway, and the monitor stops individual tasks.

Data model

Twenty-two application tables in one PostgreSQL database hold everything; LangGraph, Procrastinate and LiteLLM keep their own tables in separate schemas of the same database. Every team-owned table carries `team_id` with a row-level security policy.

Table	Key columns	Notes
`orgs`, `teams`, `users`, `memberships`	`memberships(user_id, team_id, role)`	Reused from the existing SaaS codebase
`agents`, `agent_versions`	`agent_versions(agent_id, version, spec jsonb, created_by)`	Spec is the AgentSpec YAML as JSON; versions are immutable
`skills`	`name, description, storage_path, content_hash, enabled`	Files live in object storage
`mcp_servers`, `tool_grants`	`tools_hash, status`; `grants(team_id or agent_id, server_id, tool, needs_approval)`	Hash change sets status to `needs_review`
`credentials`	`scope, provider, ciphertext, key_version`	AES-GCM; key version enables rotation
`instructions`	`scope_type, scope_id, version, body`	Org, team, project or agent layer
`tasks`	`team_id, created_by, goal, acceptance, status, parent_task_id, budget`	The ledger header
`task_steps`	`task_id, agent_id, status, depends_on, attempts, output_ref, locked_by`	Row lock is the claim for resume
`events`	`task_id, step_id, type, actor, payload jsonb, created_at`	Append-only, partitioned by month
`artifacts`	`task_id, kind, storage_path, mime, content_hash`	Code, diagrams, audio, images, video

Table	Key columns	Notes
`sources`	`url, domain, fetched_at, scores jsonb, trust_score, status`	Status: live, quarantined, superseded
`chunks`	`source_id, team_id, content, tsv tsvector, embedding vector(1024), embedding_model`	HNSW index on `embedding`, GIN on `tsv`
`memory_proposals`	`agent_id, task_id, fact, source_id, status, reviewed_by`	Pending until approved
`schedules`	`team_id, created_by, cron, task_template, next_run_at, enabled`	Runs with creator's rights
`approvals`	`step_id, tool, args jsonb, requested_at, decided_by, decision`	Backs the Approvals inbox
`usage_records`, `audit_log`	`user, team, agent, task, model, tokens_in, tokens_out, cost, latency_ms`; `actor, action, target, at`	Usage is joined with LiteLLM spend logs

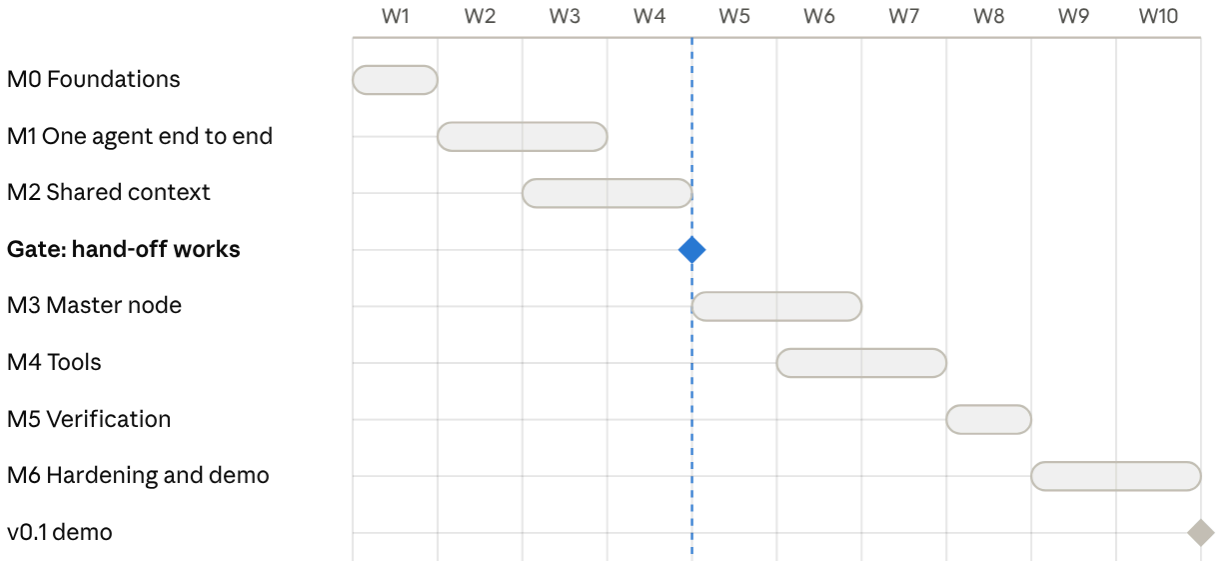
`vector(1024)` assumes bge-m3; a different embedding model means a new column or table and a background re-embed, never mixed dimensions in one index.

Build plan

Seven milestones over 10 weeks, ordered so that the riskiest idea — any agent resuming any task — is proven by the end of week 4, before routing, tools and media are layered on.

Hand-off is proven in week 4, before routing and tools are built

v0.1 build plan by week, one developer



build plan · 7 milestones over 10 weeks

Overlapping bars share a week. The week-4 gate must pass before M3 starts; if it fails, the context design is fixed before anything else is added.

What each milestone delivers

1. **M0 Foundations (week 1):** Docker Compose with Postgres + pgvector, Ollama, LiteLLM, MinIO, Caddy, API and web shell; auth and teams ported from the SaaS codebase; CI with lint, types and tests.
2. **M1 One agent end to end (weeks 2–3):** AgentSpec CRUD and versions, LangGraph executor, streaming chat, usage records for every call.
3. **M2 Shared context (weeks 3–4):** ingestion and hybrid retrieval, task ledger, event log, context compiler, resume protocol. **Gate: kill a worker mid-task and another finishes it.**
4. **M3 Master node (weeks 5–6):** manual and automatic routing, supervisor graph with delegation and review, scheduler, guards and kill switch.
5. **M4 Tools (weeks 6–7):** MCP registry, pinning and gateway; scraping pipeline with scoring and quarantine; Mermaid renderer; ElevenLabs speech and image.
6. **M5 Verification (week 8):** sandbox image, code pipeline, cross-model reviewer, research citation verifier.
7. **M6 Hardening and demo (weeks 9–10):** approvals inbox, egress proxy, audit log, usage dashboard, terminal client, evaluation sets, demo scenario.

Demo scenario for the end of M6: a member asks for "research X, write a Python module for it, document it with a diagram and a 30-second audio summary". The supervisor plans it across Researcher, Coder, Verifier, Diagrammer and Narrator agents, with every step visible, costed and verified.

Repository layout

```
platform/
├── docker-compose.yml
├── services/
│   ├── api/            # FastAPI: auth, REST, SSE (from the SaaS core)
│   ├── orchestrator/   # router, supervisor graph, monitor, scheduler
│   ├── runtime/        # agent executor, context compiler
│   ├── workers/        # Procrastinate jobs: ingestion, scraping, media
│   ├── tools/          # tool gateway, MCP client, built-in tools
│   └── sandbox/        # sandbox runner image
├── web/                # React + Vite dashboard
├── cli/                # Textual terminal client
├── agents/             # seed AgentSpec YAML files
├── skills/             # Agent Skills folders
├── infra/              # LiteLLM config, Caddy, Postgres init, MinIO
└── evals/              # routing set, hand-off tests, red-team pages
```

Definition of done for v0.1

- [ ] A member invokes an agent by `@mention` and by auto-routing, from both the web dashboard and the terminal client.
- [ ] A multi-step task runs across at least 3 agents with its ledger, events and costs visible.
- [ ] Killing a worker mid-task loses no completed step; another worker resumes from the checkpoint.
- [ ] An automated test proves team A cannot retrieve team B's knowledge.
- [ ] Every LLM and tool call appears in usage with cost; a per-task budget stop fires in a test.
- [ ] A changed MCP tool description is blocked until re-approved.
- [ ] A scraped page containing injected instructions triggers no tool call in red-team tests.
- [ ] Auto-routing reaches 85% top-1 accuracy on the 100-task labelled set.

Risks and open questions

The largest risk is that supervisor quality caps everything else, so the supervisor gets the strongest model, its own evaluation set and human approval above a cost threshold.

Risk	Mitigation	Revisit when
Supervisor plans poorly or thrashes	Strongest model, plan evaluation set, code-enforced guards, approval above cost threshold	Plan acceptance below 80% in evals
Local models fail tool calls	Schema validation with one repair retry, fallback alias, tool-heavy roles on hosted models	Local tool-call failure above 10%
GPU memory contention between local models	Cap concurrently loaded models; queue local calls	Queue wait above 30 s at p95
Procrastinate is seeking maintainers	Job layer behind an interface; Celery or Taskiq as drop-in replacement	No release for 6 months
MCP spec churn after the stateless 2026-07-28 revision	Official SDK, version negotiation, tests against old and new servers	A required server breaks
API and media costs overrun	Gateway and task budgets, cost preview, approval for video	Any team over 80% of budget mid-month
Scope creep	The v0.1 scope table is the contract; new asks go to v0.2	Every milestone review

Open questions

- [ ] Which hosted model providers, and what monthly budget per team?
- [ ] Which GPU and how much VRAM does the target machine have?
- [ ] Which teams' data may be sent to hosted APIs, and which must stay local?
- [ ] Is there an ElevenLabs plan at Pro or above? The Image & Video API rejects lower tiers.
- [ ] Which MCP servers are needed first (for example GitHub, Google Drive, Slack, a database)?
- [ ] One organisation in v0.1, or several from day one?

Sources

- Hermes Agent, Nous Research

- MCP 2026-07-28 specification release and explainer
- ElevenLabs Image & Video quickstart
- LiteLLM virtual keys and spend tracking
- Ollama OpenAI-compatible API overview
- pgvector release notes
- Procrastinate documentation
- Agent Skills open format
- Crawl4AI on PyPI
- Langfuse v3 self-hosting components
- A2A project
- OWASP Top 10 for Agentic Applications 2026
- LangGraph vs CrewAI vs AutoGen, 2026 and framework comparison incl. Microsoft Agent Framework